

Redes Neuronales Convolucionales

Redes Neuronales Convolucionales

Filtros, convolución espacial, hiperparámetros y jerarquía visual

Redes Neuronales Convolucionales

Filtros, convolución espacial, hiperparámetros y jerarquía visual

Jan Polanco Velasco

Hoja de Ruta de la Sesión

Hoja de Ruta de la Sesión

Cuatro actos que construyen la visión artificial moderna

Hoja de Ruta de la Sesión

Cuatro actos que construyen la visión artificial moderna

01

De la Imagen al Tensor

Píxeles, canales de color y por qué las redes densas (MLP) colapsan ante imágenes.

Hoja de Ruta de la Sesión

Cuatro actos que construyen la visión artificial moderna

01

De la Imagen al Tensor

Píxeles, canales de color y por qué las redes densas (MLP) colapsan ante imágenes.

02

La Operación de Convolución 2D

Filtros/kernels, detección de bordes y mapas de características (*feature maps*).

Hoja de Ruta de la Sesión

Cuatro actos que construyen la visión artificial moderna

01

De la Imagen al Tensor

Píxeles, canales de color y por qué las redes densas (MLP) colapsan ante imágenes.

02

La Operación de Convolución 2D

Filtros/kernels, detección de bordes y mapas de características (*feature maps*).

03

Geometría de la Convolución

Padding (*Valid vs. Same*), Stride y Convolución sobre tensores multicanal 3D.

Hoja de Ruta de la Sesión

Cuatro actos que construyen la visión artificial moderna

01

De la Imagen al Tensor

Píxeles, canales de color y por qué las redes densas (MLP) colapsan ante imágenes.

02

La Operación de Convolución 2D

Filtros/kernels, detección de bordes y mapas de características (*feature maps*).

03

Geometría de la Convolución

Padding (*Valid vs. Same*), Stride y Convolución sobre tensores multicanal 3D.

04

Pooling y Jerarquía de Abstracción

Invarianza a traslaciones, campo receptivo y la arquitectura canónica en Keras.

01

01

De la Imagen al Tensor

01

De la Imagen al Tensor

Representación matricial y el colapso de las redes densas (MLP)

¿Qué es una Imagen para una Máquina?

¿Qué es una Imagen para una Máquina?

Tensores discretos de intensidad

¿Qué es una Imagen para una Máquina?

Tensores discretos de intensidad

- **Escala de Grises (2D):** Una matriz de dimensiones

$H \times W$, donde cada valor representa

luminosidad:

$$I(x, y) \in [0, 255] \quad (\text{o normalizado en } [0.0, 1.0])$$

¿Qué es una Imagen para una Máquina?

Tensores discretos de intensidad

- **Escala de Grises (2D):** Una matriz de dimensiones

$H \times W$, donde cada valor representa

luminosidad:

$$I(x, y) \in [0, 255] \quad (\text{o normalizado en } [0.0, 1.0])$$

- **Color RGB (3D):** Tres matrices apiladas en canales

(Rojo, Verde, Azul):

$$\mathbf{X} \in \mathbb{R}^{H \times W \times 3}$$

¿Qué es una Imagen para una Máquina?

Tensores discretos de intensidad

- **Escala de Grises (2D):** Una matriz de dimensiones

$H \times W$, donde cada valor representa
luminosidad:

$$I(x, y) \in [0, 255] \quad (\text{o normalizado en } [0.0, 1.0])$$

- **Color RGB (3D):** Tres matrices apiladas en canales
(Rojo, Verde, Azul):

$$\mathbf{X} \in \mathbb{R}^{H \times W \times 3}$$

- **La Vecindad Espacial:** Un píxel aislado carece de significado semántico; su valor solo tiene sentido en relación con sus **vecinos contiguos**.

¿Qué es una Imagen para una Máquina?

Tensores discretos de intensidad

- **Escala de Grises (2D):** Una matriz de dimensiones $H \times W$, donde cada valor representa luminosidad:

$$I(x, y) \in [0, 255] \quad (\text{o normalizado en } [0.0, 1.0])$$

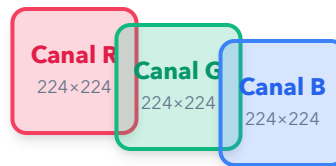
- **Color RGB (3D):** Tres matrices apiladas en canales (*Rojo, Verde, Azul*):

$$\mathbf{X} \in \mathbb{R}^{H \times W \times 3}$$

- **La Vecindad Espacial:** Un píxel aislado carece de significado semántico; su valor solo tiene sentido en relación con sus **vecinos contiguos**.

Estructura Tensor RGB:

[Alto, Ancho, Canales]



Una imagen típica de smartphone (12 MP) contiene más de **36 millones de valores escalares**.

El Dilema de la MLP en Visión

¿Por qué aplanar (*Flatten*) destruye la imagen?

El Dilema de la MLP en Visión

¿Por qué aplanar (*Flatten*) destruye la imagen?

- **1. Pérdida de la Topología Espacial:** Al vectorizar una imagen $2D \rightarrow 1D$, el píxel (x, y) pierde su cercanía física con $(x, y + 1)$, destruyendo formas, texturas y contornos.

El Dilema de la MLP en Visión

¿Por qué aplanar (*Flatten*) destruye la imagen?

- **1. Pérdida de la Topología Espacial:** Al vectorizar una imagen $2D \rightarrow 1D$, el píxel (x, y) pierde su cercanía física con $(x, y + 1)$, destruyendo formas, texturas y contornos.
- **2. Explosión Inmanejable de Parámetros:**
 - Imagen modesta: $200 \times 200 \times 3 = 120.000$ entradas.
 - Primera capa oculta con 1.000 neuronas:

$$120.000 \times 1.000 = \mathbf{120} \text{ millones de pesos}$$

- Resultado: sobreajuste garantizado y lentitud extrema.

El Dilema de la MLP en Visión

¿Por qué aplanar (*Flatten*) destruye la imagen?

- **1. Pérdida de la Topología Espacial:** Al vectorizar una imagen $2D \rightarrow 1D$, el píxel (x, y) pierde su cercanía física con $(x, y + 1)$, destruyendo formas, texturas y contornos.
- **2. Explosión Inmanejable de Parámetros:**
 - Imagen modesta: $200 \times 200 \times 3 = 120.000$ entradas.
 - Primera capa oculta con 1.000 neuronas:

$$120.000 \times 1.000 = \mathbf{120} \text{ millones de pesos}$$

- Resultado: sobreajuste garantizado y lentitud extrema.
- **3. Cero Invarianza a Traslación:** Si un gato se desplaza 5 píxeles a la derecha, para una MLP es un vector totalmente nuevo y desconectado.

El Dilema de la MLP en Visión

¿Por qué aplanar (*Flatten*) destruye la imagen?

- **1. Pérdida de la Topología Espacial:** Al vectorizar una imagen $2D \rightarrow 1D$, el píxel (x, y) pierde su cercanía física con $(x, y + 1)$, destruyendo formas, texturas y contornos.
- **2. Explosión Inmanejable de Parámetros:**
 - Imagen modesta: $200 \times 200 \times 3 = 120.000$ entradas.
 - Primera capa oculta con 1.000 neuronas:

$$120.000 \times 1.000 = \mathbf{120 \text{ millones de pesos}}$$

- Resultado: sobreajuste garantizado y lentitud extrema.
- **3. Cero Invarianza a Traslación:** Si un gato se desplaza 5 píxeles a la derecha, para una MLP es un vector totalmente nuevo y desconectado.

⚠ El Fracaso del "Flatten" en MLP

Matriz (3×3):

[A, B, C]

[D, E, F] → Vector: [A, B, C, D, E, F, G, H, I]

[G, H, I]

¡B y E eran vecinos verticales, ahora están separados!

Necesitamos una operación que preserve la grilla bidimensional y aproveche la correlación local: la **Convolución**.

02

02

La Operación de Convolución 2D

02

La Operación de Convolución 2D

Filtros locales, detección de características y peso compartido

Sobel Vertical

Laplaciano

Identidad

Entrada X (5×5)

10	10	10	200	200
10	10	10	200	200
10	10	10	200	200
10	10	10	200	200
10	10	10	200	200

Filtro (3×3)

-1	0	1
-2	0	2
-1	0	1

Salida (3×3)

0	760	760
0	760	760
0	760	760

Posición Kernel (r, c): (1, 1)



Sobel Vertical (Detecta bordes verticales)

$$\sum (X_{\text{local}} \odot K) = (10 \times -1) + (200 \times 1) + (10 \times -2) + (200 \times 2) + \dots$$

La Mecánica del Kernel Convolutivo

Sobel Vertical

Laplaciano

Identidad

Entrada X (5x5)

10	10	10	200	200
10	10	10	200	200
10	10	10	200	200
10	10	10	200	200
10	10	10	200	200

Filtro (3x3)

-1	0	1
-2	0	2
-1	0	1

Salida (3x3)

0	760	760
0	760	760
0	760	760

Posición Kernel (r, c): (1, 1)

Sobel Vertical (Detecta bordes verticales)

$\sum (X_{\text{local}} \odot K) = (10 \times -1) + (200 \times 1) + (10 \times -2) + (200 \times 2) + \dots$

La Mecánica del Kernel Convolutacional

Producto punto deslizante

Sobel Vertical

Laplaciano

Identidad

Entrada X (5×5)

10	10	10	200	200
10	10	10	200	200
10	10	10	200	200
10	10	10	200	200
10	10	10	200	200

Filtro (3×3)

-1	0	1
-2	0	2
-1	0	1

Salida (3×3)

0	760	760
0	760	760
0	760	760

Posición Kernel (r, c): (1, 1)

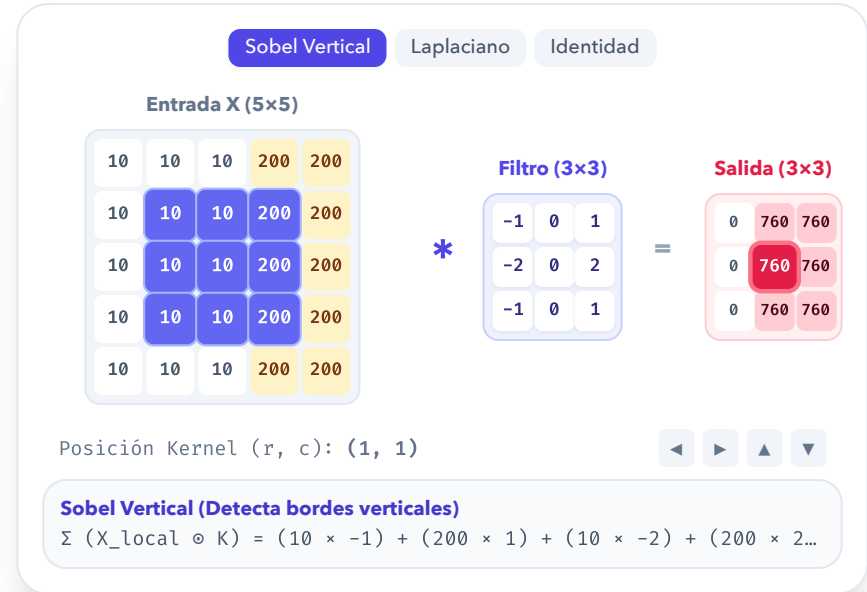
Sobel Vertical (Detecta bordes verticales)

$\sum (X_{\text{local}} \odot K) = (10 \times -1) + (200 \times 1) + (10 \times -2) + (200 \times 2) + (10 \times -1) + (200 \times 1)$

La Mecánica del Kernel Convolucional

Producto punto deslizante

- **Kernel ($\mathbf{K}$):** matriz pequeña de pesos entrenables (3×3 o 5×5).



La Mecánica del Kernel Convolucional

Producto punto deslizante

- **Kernel (K):** matriz pequeña de pesos entrenables (3×3 o 5×5).
- **Operación local:** superpone el filtro, multiplica elemento a elemento y suma + sesgo b :

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I_{i+m, j+n} K_{m, n} + b$$

Sobel Vertical

Laplaciano

Identidad

Entrada X (5×5)

10	10	10	200	200
10	10	10	200	200
10	10	10	200	200
10	10	10	200	200
10	10	10	200	200

Filtro (3×3)

-1	0	1
-2	0	2
-1	0	1

Salida (3×3)

0	760	760
0	760	760
0	760	760

Posición Kernel (r, c): (1, 1)

Sobel Vertical (Detecta bordes verticales)

$\sum (X_{\text{local}} \odot K) = (10 \times -1) + (200 \times 1) + (10 \times -2) + (200 \times 2) + \dots$

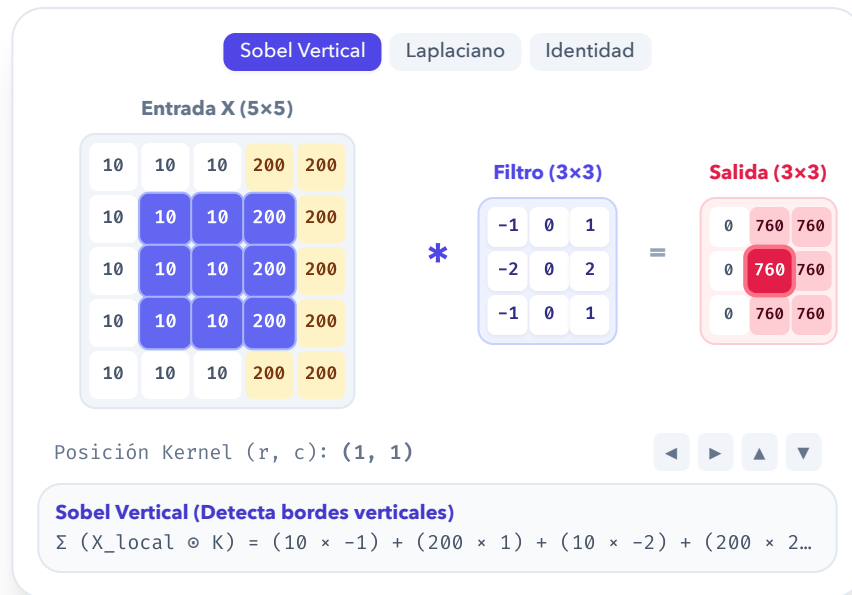
La Mecánica del Kernel Convolutivo

Producto punto deslizante

- **Kernel (K):** matriz pequeña de pesos entrenables (3×3 o 5×5).
- **Operación local:** superpone el filtro, multiplica elemento a elemento y suma + sesgo b :

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I_{i+m, j+n} K_{m, n} + b$$

- **Dos principios:**
 1. Conexiones locales: cada neurona ve solo un parche diminuto.
 2. **Pesos compartidos:** el mismo filtro barre **toda** la imagen.



Filtros Clásicos vs. Filtros Aprendidos

De la visión artificial artesanal al Deep
Learning

Filtros Clásicos vs. Filtros Aprendidos

De la visión artificial artesanal al Deep Learning

- **Décadas de 1970 - 1990 (Ingeniería Manual):** Los investigadores pasaban años diseñando matrices fijas a mano para detectar bordes o esquinas:
 - **Sobel Horizontal:** Detecta cambios abruptos de brillo vertical.
 - **Sobel Vertical:** Detecta cambios de brillo horizontal.

Filtros Clásicos vs. Filtros Aprendidos

De la visión artificial artesanal al Deep Learning

- **Décadas de 1970 - 1990 (Ingeniería Manual):** Los investigadores pasaban años diseñando matrices fijas a mano para detectar bordes o esquinas:
 - **Sobel Horizontal:** Detecta cambios abruptos de brillo vertical.
 - **Sobel Vertical:** Detecta cambios de brillo horizontal.
- **La Revolución de las CNNs (LeCun, Krizhevsky):**
 - Los pesos del kernel **no se fijan a mano**.
 - Son parámetros libres inicializados aleatoriamente que se optimizan con **Backpropagation y Gradient Descent**.
 - La red descubre por sí misma cuáles son los detectores visuales óptimos para la tarea.

Filtros Clásicos vs. Filtros Aprendidos

De la visión artificial artesanal al Deep Learning

- **Décadas de 1970 - 1990 (Ingeniería Manual):** Los investigadores pasaban años diseñando matrices fijas a mano para detectar bordes o esquinas:
 - **Sobel Horizontal:** Detecta cambios abruptos de brillo vertical.
 - **Sobel Vertical:** Detecta cambios de brillo horizontal.
- **La Revolución de las CNNs (LeCun, Krizhevsky):**
 - Los pesos del kernel **no se fijan a mano**.
 - Son parámetros libres inicializados aleatoriamente que se optimizan con **Backpropagation** y **Gradient Descent**.
 - La red descubre por sí misma cuáles son los detectores visuales óptimos para la tarea.

Kernels Clásicos de Visión (Sobel):

Borde Vertical

$$\begin{bmatrix} -1, & 0, & +1 \\ -2, & 0, & +2 \\ -1, & 0, & +1 \end{bmatrix}$$

Borde Horizontal

$$\begin{bmatrix} +1, & +2, & +1 \\ 0, & 0, & 0 \\ -1, & -2, & -1 \end{bmatrix}$$

En una capa 'Conv2D(32, 3)', la red aprende simultáneamente **32 filtros distintos** como estos en paralelo.

03

03

Geometría de la Convolución

03

Geometría de la Convolución

Padding, Stride y Convolución sobre Volúmenes 3D

El Efecto de Reducción y el Padding

El Efecto de Reducción y el Padding

Protegiendo los bordes de la imagen

El Efecto de Reducción y el Padding

Protegiendo los bordes de la imagen

- La **convolución cruda encoge**: $N \times N$ con kernel $f \times f$ da:

$$O = N - f + 1 \quad (5 \times 5, f = 3 \rightarrow \mathbf{3 \times 3})$$

El Efecto de Reducción y el Padding

Protegiendo los bordes de la imagen

- La **convolución cruda encoge**: $N \times N$ con kernel $f \times f$ da:

$$O = N - f + 1 \quad (5 \times 5, f = 3 \rightarrow 3 \times 3)$$

- Las esquinas participan una sola vez: se pierde información perimetral.

El Efecto de Reducción y el Padding

Protegiendo los bordes de la imagen

- La convolución cruda **encoge**: $N \times N$ con kernel $f \times f$ da:

$$O = N - f + 1 \quad (5 \times 5, f = 3 \rightarrow 3 \times 3)$$

- Las esquinas participan una sola vez: se pierde información perimetral.
- Solución: Padding (p)** con ceros (*Zero Padding*):
 - Valid ($p = 0$): sin relleno, encoge.
 - Same: $p = \frac{f-1}{2}$ ceros, la salida conserva el tamaño.

El Efecto de Reducción y el Padding

Protegiendo los bordes de la imagen

- La convolución cruda encoge: $N \times N$ con kernel $f \times f$ da:

$$O = N - f + 1 \quad (5 \times 5, f = 3 \rightarrow 3 \times 3)$$

- Las esquinas participan una sola vez: se pierde información perimetral.
- Solución: Padding (p)** con ceros (*Zero Padding*):
 - Valid ($p = 0$): sin relleno, encoge.
 - Same:** $p = \frac{f-1}{2}$ ceros, la salida conserva el tamaño.

Same Padding (Filtro 3x3, p=1)



0	0	0	0	0	0	0
0	X	X	X	X	X	0
0	X	X	X	X	X	0
0	X	X	X	X	X	0
0	X	X	X	X	X	0
0	X	X	X	X	X	0
0	0	0	0	0	0	0

La cuadrícula de ceros (en gris) permite centrar el filtro en los bordes originales sin perder resolución espacial.

Stride y la Ecuación Dimensional Universal

¿A qué velocidad se desplaza el kernel?

Stride y la Ecuación Dimensional Universal

¿A qué velocidad se desplaza el kernel?

- **Stride (s):** El tamaño del salto que da el kernel en cada paso (horizontal y vertical).
 - $s = 1$: Paso suave, máxima cobertura y resolución.
 - $s = 2$: Salta de 2 en 2 píxeles, reduciendo el mapa a la **mitad** de su tamaño (submuestreo incorporado).

Stride y la Ecuación Dimensional Universal

¿A qué velocidad se desplaza el kernel?

- **Stride (s):** El tamaño del salto que da el kernel en cada paso (horizontal y vertical).
 - $s = 1$: Paso suave, máxima cobertura y resolución.
 - $s = 2$: Salta de 2 en 2 píxeles, reduciendo el mapa a la **mitad** de su tamaño (submuestreo incorporado).
- **Fórmula de Dimensionalidad Universal:**

$$O = \left\lfloor \frac{N + 2p - f}{s} \right\rfloor + 1$$

Stride y la Ecuación Dimensional Universal

¿A qué velocidad se desplaza el kernel?

- **Stride (s):** El tamaño del salto que da el kernel en cada paso (horizontal y vertical).
 - $s = 1$: Paso suave, máxima cobertura y resolución.
 - $s = 2$: Salta de 2 en 2 píxeles, reduciendo el mapa a la **mitad** de su tamaño (submuestreo incorporado).
- **Fórmula de Dimensionalidad Universal:**

$$O = \left\lfloor \frac{N + 2p - f}{s} \right\rfloor + 1$$

- **Ejemplo Práctico:**
 - Entrada: $N = 28$ (MNIST)
 - Filtro: $f = 3$ | Padding: $p = 1$ (*same*) | Stride: $s = 2$

$$O = \left\lfloor \frac{28 + 2(1) - 3}{2} \right\rfloor + 1 = \left\lfloor \frac{27}{2} \right\rfloor + 1 = 13 + 1 = \mathbf{14}$$

Stride y la Ecuación Dimensional Universal

¿A qué velocidad se desplaza el kernel?

- **Stride (s):** El tamaño del salto que da el kernel en cada paso (horizontal y vertical).
 - $s = 1$: Paso suave, máxima cobertura y resolución.
 - $s = 2$: Salta de 2 en 2 píxeles, reduciendo el mapa a la **mitad** de su tamaño (submuestreo incorporado).
- **Fórmula de Dimensionalidad Universal:**

$$O = \left\lfloor \frac{N + 2p - f}{s} \right\rfloor + 1$$

- **Ejemplo Práctico:**
 - Entrada: $N = 28$ (MNIST)
 - Filtro: $f = 3$ | Padding: $p = 1$ (*same*) | Stride: $s = 2$

$$O = \left\lfloor \frac{28 + 2(1) - 3}{2} \right\rfloor + 1 = \left\lfloor \frac{27}{2} \right\rfloor + 1 = 13 + 1 = 14$$

Efecto del Stride

Control Espacial

Stride = 1:

Mantiene el detalle denso de las texturas. Es el estándar en las primeras capas.

Stride ≥ 2 (Strided Convolutions):

Alternativa moderna y aprendible a las capas de Pooling para comprimir dimensiones.

Convolución en Volúmenes 3D

Múltiples canales de entrada y
múltiples filtros de salida

Convolución en Volúmenes 3D

Múltiples canales de entrada y múltiples filtros de salida

- Los canales del filtro **DEBEN** coincidir con la entrada: Si la entrada es una imagen RGB ($H \times W \times \mathbf{3}$), cada filtro individual tiene volumen:

$$\mathbf{K}_i \in \mathbb{R}^{f \times f \times \mathbf{3}}$$

Convolución en Volúmenes 3D

Múltiples canales de entrada y múltiples filtros de salida

- **Los canales del filtro DEBEN coincidir con la entrada:** Si la entrada es una imagen RGB ($H \times W \times \mathbf{3}$), cada filtro individual tiene volumen:

$$\mathbf{K}_i \in \mathbb{R}^{f \times f \times \mathbf{3}}$$

- **Cada filtro produce UN SOLO mapa 2D:** Se convoluciona cada canal 2D por separado, se suman los 3 resultados punto a punto y se añade un sesgo escalar b_i .

Convolución en Volúmenes 3D

Múltiples canales de entrada y múltiples filtros de salida

- **Los canales del filtro DEBEN coincidir con la entrada:** Si la entrada es una imagen RGB ($H \times W \times \mathbf{3}$), cada filtro individual tiene volumen:

$$\mathbf{K}_i \in \mathbb{R}^{f \times f \times \mathbf{3}}$$

- **Cada filtro produce UN SOLO mapa 2D:** Se convoluciona cada canal 2D por separado, se suman los 3 resultados punto a punto y se añade un sesgo escalar b_i .
- **Banco de K Filtros:** Si usamos K filtros distintos en una capa:

$$\text{Entrada } (H \times W \times C) \xrightarrow{K \text{ filtros}} \text{Salida } (H' \times W' \times \mathbf{K})$$

Convolución en Volúmenes 3D

Múltiples canales de entrada y múltiples filtros de salida

- **Los canales del filtro DEBEN coincidir con la entrada:** Si la entrada es una imagen RGB ($H \times W \times \mathbf{3}$), cada filtro individual tiene volumen:

$$\mathbf{K}_i \in \mathbb{R}^{f \times f \times \mathbf{3}}$$

- **Cada filtro produce UN SOLO mapa 2D:** Se convoluciona cada canal 2D por separado, se suman los 3 resultados punto a punto y se añade un sesgo escalar b_i .
- **Banco de K Filtros:** Si usamos K filtros distintos en una capa:

$$\text{Entrada } (H \times W \times C) \xrightarrow{K \text{ filtros}} \text{Salida } (H' \times W' \times \mathbf{K})$$

- La profundidad de la salida **no depende** del número de canales originales, sino de cuántos filtros configuramos en Keras.

Convolución en Volúmenes 3D

Múltiples canales de entrada y múltiples filtros de salida

- Los canales del filtro **DEBEN** coincidir con la entrada: Si la entrada es una imagen RGB ($H \times W \times 3$), cada filtro individual tiene volumen:

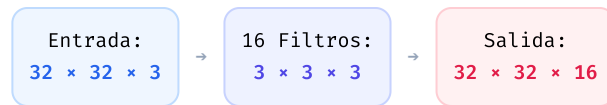
$$\mathbf{K}_i \in \mathbb{R}^{f \times f \times 3}$$

- Cada filtro produce **UN SOLO** mapa 2D: Se convoluciona cada canal 2D por separado, se suman los 3 resultados punto a punto y se añade un sesgo escalar b_i .
- Banco de K Filtros: Si usamos K filtros distintos en una capa:

$$\text{Entrada } (H \times W \times C) \xrightarrow{K \text{ filtros}} \text{Salida } (H' \times W' \times \mathbf{K})$$

- La profundidad de la salida **no depende** del número de canales originales, sino de cuántos filtros configuramos en Keras.

Transformación de Volúmenes:



Total parámetros por filtro: $(3 \times 3 \times 3) + 1 \text{ bias} = 28$
Capa completa (16 filtros): $28 \times 16 = 448$ parámetros

04

04

Pooling, Jerarquía y Arquitectura CNN

04

Pooling, Jerarquía y Arquitectura CNN

Invarianza espacial, campo receptivo y el flujo completo

Capas de Pooling (Submuestreo)

Resumiendo la presencia de
características

Capas de Pooling (Submuestreo)

Resumiendo la presencia de
características

- **Objetivo:** Reducir agresivamente el ancho y alto del tensor sin modificar la cantidad de canales.

Capas de Pooling (Submuestreo)

Resumiendo la presencia de características

- **Objetivo:** Reducir agresivamente el ancho y alto del tensor sin modificar la cantidad de canales.
- **Max Pooling ($2 \times 2, s = 2$):**
 - Toma el valor más alto en cada bloque 2×2 .
 - Pregunta semántica: "*¿Estuvo presente esta característica en la región?*"
 - Reduce el área espacial en un **75%** (cada lado a la mitad).

Capas de Pooling (Submuestreo)

Resumiendo la presencia de características

- **Objetivo:** Reducir agresivamente el ancho y alto del tensor sin modificar la cantidad de canales.
- **Max Pooling ($2 \times 2, s = 2$):**
 - Toma el valor más alto en cada bloque 2×2 .
 - Pregunta semántica: "*¿Estuvo presente esta característica en la región?*"
 - Reduce el área espacial en un **75%** (cada lado a la mitad).
- **Propiedad Fundamental: Invarianza Local a Traslaciones:** Si un borde o textura se mueve 1 o 2 píxeles dentro de la ventana de pooling, el valor máximo resultante **permanece idéntico**.

Capas de Pooling (Submuestreo)

Resumiendo la presencia de características

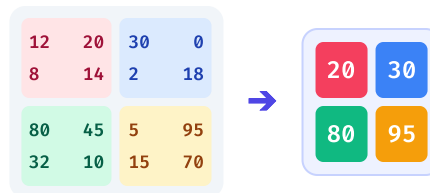
- **Objetivo:** Reducir agresivamente el ancho y alto del tensor sin modificar la cantidad de canales.
- **Max Pooling ($2 \times 2, s = 2$):**
 - Toma el valor más alto en cada bloque 2×2 .
 - Pregunta semántica: "*¿Estuvo presente esta característica en la región?*"
 - Reduce el área espacial en un **75%** (cada lado a la mitad).
- **Propiedad Fundamental: Invarianza Local a Traslaciones:** Si un borde o textura se mueve 1 o 2 píxeles dentro de la ventana de pooling, el valor máximo resultante **permanece idéntico**.
- **Ventaja adicional:** No añade ningún parámetro entrenable a la red.

Capas de Pooling (Submuestreo)

Resumiendo la presencia de características

- **Objetivo:** Reducir agresivamente el ancho y alto del tensor sin modificar la cantidad de canales.
- **Max Pooling (2×2 , $s = 2$):**
 - Toma el valor más alto en cada bloque 2×2 .
 - Pregunta semántica: "*¿Estuvo presente esta característica en la región?*"
 - Reduce el área espacial en un **75%** (cada lado a la mitad).
- **Propiedad Fundamental: Invarianza Local a Traslaciones:** Si un borde o textura se mueve 1 o 2 píxeles dentro de la ventana de pooling, el valor máximo resultante **permanece idéntico**.
- **Ventaja adicional:** No añade ningún parámetro entrenable a la red.

Mecánica de Max Pooling 2×2 (Stride 2):



Se conserva el valor dominante de cada cuadrante eliminando detalles irrelevantes de fondo.

La Jerarquía de Características Visuales

La Jerarquía de Características Visuales

Cómo "ve" el mundo una red neuronal convolucional profunda

La Jerarquía de Características Visuales

Cómo "ve" el mundo una red neuronal convolucional profunda

Capas Iniciales

Características Primitivas

- Bordes orientados (verticales, diagonales).
- Gradientes de luz y sombras.
- Parches simples de color uniforme.

Campo Receptivo: Pequeño (Local)

La Jerarquía de Características Visuales

Cómo "ve" el mundo una red neuronal convolucional profunda

Capas Iniciales

Características Primitivas

- Bordes orientados (verticales, diagonales).
- Gradientes de luz y sombras.
- Parches simples de color uniforme.

Campo Receptivo: Pequeño (Local)

Capas Intermedias

Partes de Objetos y Texturas

- Combinación de bordes: curvas, esquinas.
- Texturas repetitivas (rayas, mallas).
- Partes anatómicas: ojos, narices, ruedas.

Campo Receptivo: Mediano

La Jerarquía de Características Visuales

Cómo "ve" el mundo una red neuronal convolucional profunda

Capas Iniciales

Características Primitivas

- Bordes orientados (verticales, diagonales).
- Gradientes de luz y sombras.
- Parches simples de color uniforme.

Campo Receptivo: Pequeño (Local)

Capas Intermedias

Partes de Objetos y Texturas

- Combinación de bordes: curvas, esquinas.
- Texturas repetitivas (rayas, mallas).
- Partes anatómicas: ojos, narices, ruedas.

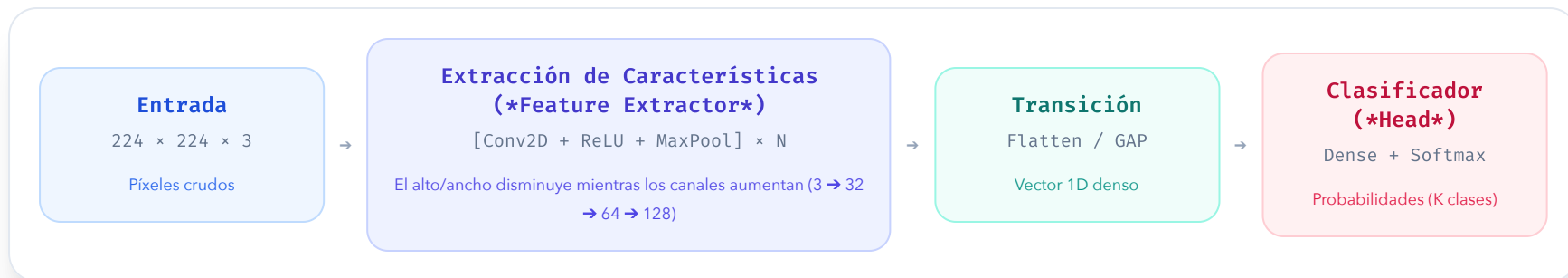
Campo Receptivo: Mediano

Capas Profundas

Conceptos Semánticos Globales

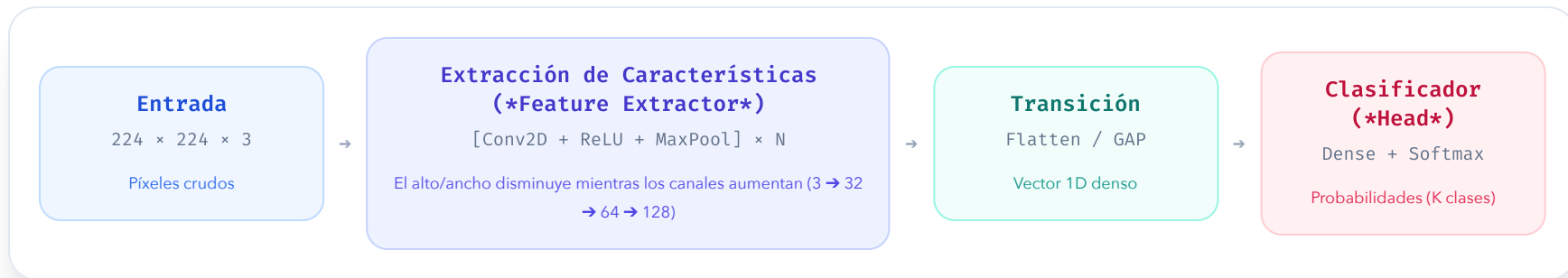
- Rostros humanos completos.
- Siluetas de vehículos, animales, muebles.
- Representaciones de alta abstracción listas para clasificar.

Campo Receptivo: Global (Toda la imagen)



Regla de Diseño Estándar: A medida que la red profundiza, la resolución espacial disminuye ($H \downarrow, W \downarrow$) mediante MaxPooling, mientras que la riqueza semántica aumenta incrementando los filtros ($C \uparrow$).

Anatomía de la Arquitectura Canónica CNN



Regla de Diseño Estándar: A medida que la red profundiza, la resolución espacial disminuye ($H \downarrow, W \downarrow$) mediante MaxPooling, mientras que la riqueza semántica aumenta incrementando los filtros ($C \uparrow$).

```
# Paso 1: Configurar la capa de entrada convolucional
model = Sequential([
    Input(shape=(28, 28, 1)),
    Conv2D(filters=32, kernel_size=(3, 3), activation='relu', padding='same')
])
```

Implementación en Keras con Shiki Magic Move

```
# Paso 1: Configurar la capa de entrada convolucional
model = Sequential([
    Input(shape=(28, 28, 1)),
    Conv2D(filters=32, kernel_size=(3, 3), activation='relu', padding='same')
])
```

Implementación en Keras con Shiki Magic Move

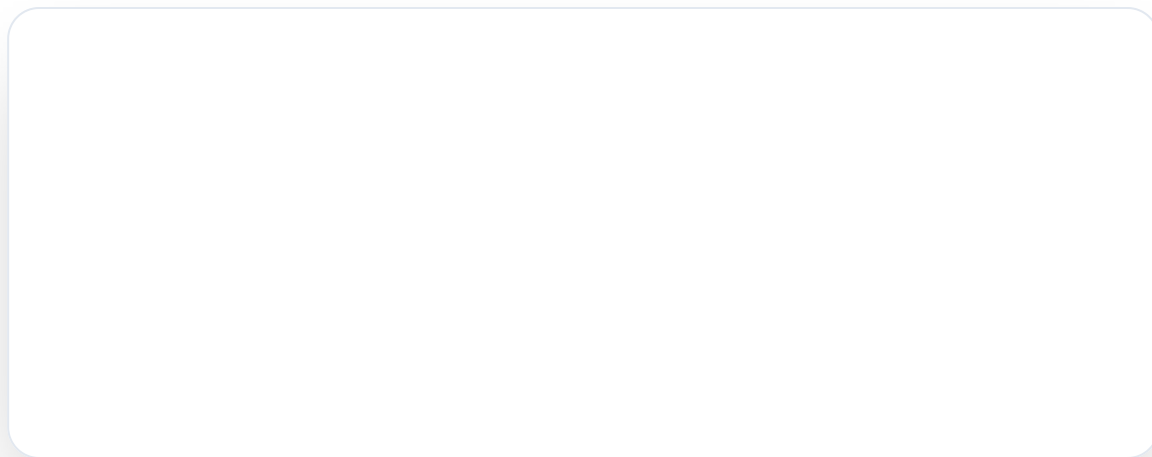
```
# Paso 2: Agregar compresión espacial con Max Pooling
model = Sequential([
    Input(shape=(28, 28, 1)),
    Conv2D(filters=32, kernel_size=(3, 3), activation='relu', padding='same'),
    MaxPooling2D(pool_size=(2, 2)), # Reduce de 28×28 a 14×14
    Conv2D(filters=64, kernel_size=(3, 3), activation='relu', padding='same'),
    MaxPooling2D(pool_size=(2, 2)) # Reduce de 14×14 a 7×7
])
```

Implementación en Keras con Shiki Magic Move

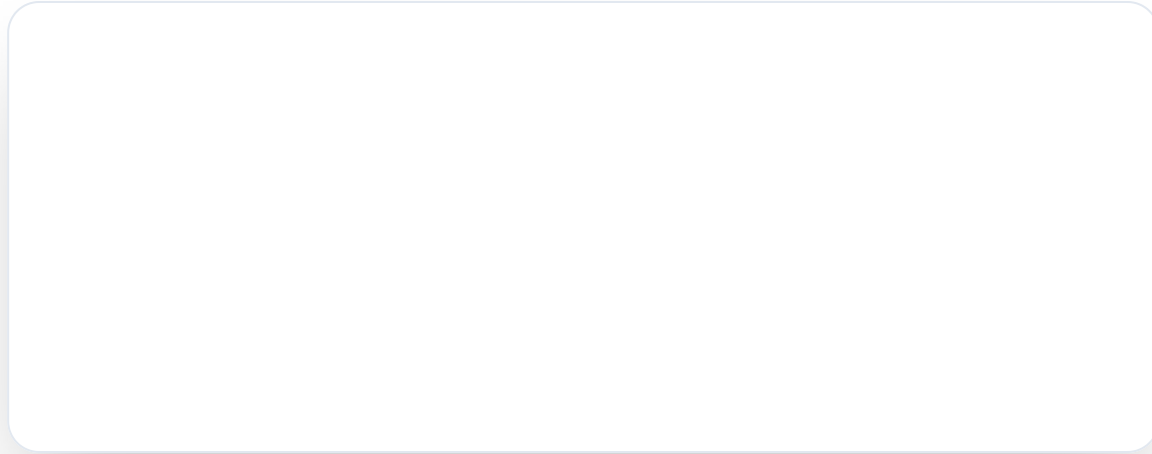
```
# Paso 3: Conectar el extractor con el clasificador denso
model = Sequential([
    Input(shape=(28, 28, 1)),
    Conv2D(filters=32, kernel_size=(3, 3), activation='relu', padding='same'),
    MaxPooling2D(pool_size=(2, 2)),
    Conv2D(filters=64, kernel_size=(3, 3), activation='relu', padding='same'),
    MaxPooling2D(pool_size=(2, 2)),
    Flatten(),                    # Vectoriza 7 x 7 x 64 = 3.136 características
    Dropout(0.4),                 # Regularización contra overfitting
    Dense(128, activation='relu'),
    Dense(10, activation='softmax') # 10 clases de salida
])
```

Implementación en Keras con Shiki Magic Move

```
# Paso 4: Compilación y optimización
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
model.summary()
```



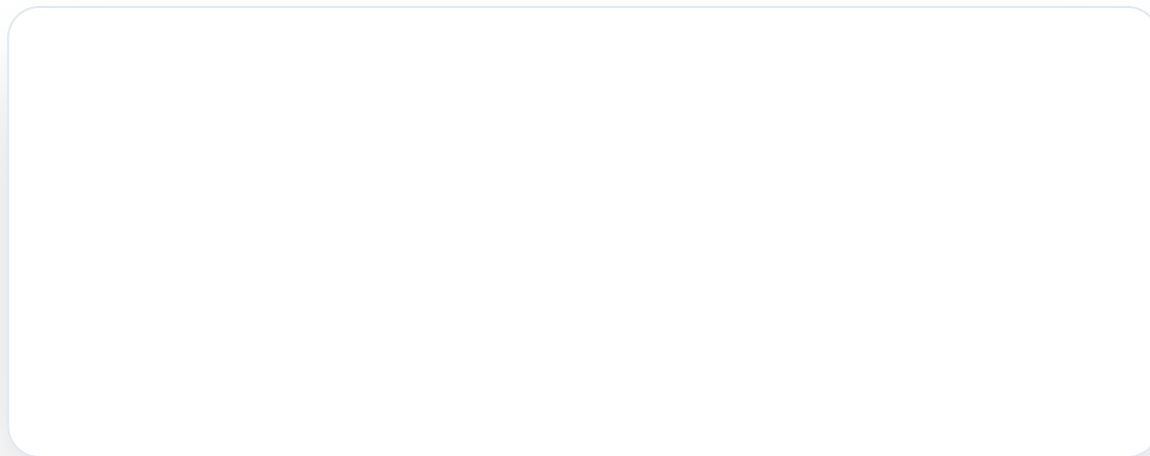
Laboratorio Práctico de la Sesión 4





Laboratorio Práctico de la Sesión 4

Clasificación de Imágenes y Extracción de Mapas de Activación





Laboratorio Práctico de la Sesión 4

Clasificación de Imágenes y Extracción de Mapas de Activación

1

Visualización de Convoluciones Manuales

Aplicación de filtros Sobel y Laplaciano con NumPy y OpenCV en imágenes reales.



Laboratorio Práctico de la Sesión 4

Clasificación de Imágenes y Extracción de Mapas de Activación

1

Visualización de Convoluciones Manuales

Aplicación de filtros Sobel y Laplaciano con NumPy y OpenCV en imágenes reales.

2

Entrenamiento de una CNN en Keras

Construcción de arquitectura Conv2D + BatchNorm + MaxPooling sobre CIFAR-10 / Fashion-MNIST.



Laboratorio Práctico de la Sesión 4

Clasificación de Imágenes y Extracción de Mapas de Activación

1

Visualización de Convoluciones Manuales

Aplicación de filtros Sobel y Laplaciano con NumPy y OpenCV en imágenes reales.

2

Entrenamiento de una CNN en Keras

Construcción de arquitectura Conv2D + BatchNorm + MaxPooling sobre CIFAR-10 / Fashion-MNIST.

3

Inspección de Feature Maps Internos

Extracción y visualización de lo que las neuronas aprendieron en cada capa intermedia.

Lo que aprendimos hoy

Lo que aprendimos hoy

Topología Espacial

Aplanar una imagen destruye la vecindad. Las CNNs preservan la grilla y reducen millones de parámetros gracias a los pesos compartidos.

Lo que aprendimos hoy

Topología Espacial

Aplanar una imagen destruye la vecindad. Las CNNs preservan la grilla y reducen millones de parámetros gracias a los pesos compartidos.

Convolución y Filtros

El kernel realiza un producto punto local. La red aprende los filtros automáticamente mediante descenso de gradiente.

Lo que aprendimos hoy

Topología Espacial

Aplanar una imagen destruye la vecindad. Las CNNs preservan la grilla y reducen millones de parámetros gracias a los pesos compartidos.

Convolución y Filtros

El kernel realiza un producto punto local. La red aprende los filtros automáticamente mediante descenso de gradiente.

Padding y Stride

El padding protege los bordes y preserva dimensiones (*same*), mientras que el stride controla la tasa de salto espacial.

Lo que aprendimos hoy

Topología Espacial

Aplanar una imagen destruye la vecindad. Las CNNs preservan la grilla y reducen millones de parámetros gracias a los pesos compartidos.

Convolución y Filtros

El kernel realiza un producto punto local. La red aprende los filtros automáticamente mediante descenso de gradiente.

Padding y Stride

El padding protege los bordes y preserva dimensiones (*same*), mientras que el stride controla la tasa de salto espacial.

Pooling y Jerarquía

Max Pooling aporta invarianza a pequeñas traslaciones. Las capas tempranas detectan bordes y las profundas ensamblan objetos complejos.

Lo que aprendimos hoy

Topología Espacial

Aplanar una imagen destruye la vecindad. Las CNNs preservan la grilla y reducen millones de parámetros gracias a los pesos compartidos.

Convolución y Filtros

El kernel realiza un producto punto local. La red aprende los filtros automáticamente mediante descenso de gradiente.

Padding y Stride

El padding protege los bordes y preserva dimensiones (*same*), mientras que el stride controla la tasa de salto espacial.

Pooling y Jerarquía

Max Pooling aporta invarianza a pequeñas traslaciones. Las capas tempranas detectan bordes y las profundas ensamblan objetos complejos.

DEEP LEARNING · Pontificia Universidad Javeriana Cali
Jan Polanco Velasco